

ОТВЕТЫ НА ЭКЗАМЕНАЦИОННЫЕ ВОПРОСЫ

Технология программирования в Интернет

2 курс, ПИ • 2026

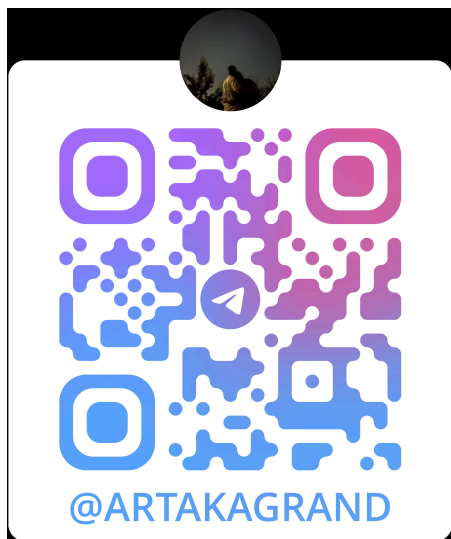
Составил: Мышковец Артём 2-10

tg: @artakagrand

t.me/artakagrand

(Для связи скопировать и вставить в браузер)

или



Предисловие !!! ОБЯЗАТЕЛЬНО К ПРОЧТЕНИЮ !!!

Друзья, этот файл составлен мной на чистом энтузиазме с бюджетом в пару кружек кофе. Для его составления использовался Claude и Gemini. Источниками были взяты самописные конспекты лекций, электронные конспекты от самого Смелова, выполненные лабораторные работы, а так же собственные знания.

Ответы писались и редактировались мной в надежде помочь самому себе успешно сдать экзамен, а так же помочь в этом всем, кто это читает. Если вам понравилось или помогло содержимое этого файла, то можете написать мне в тг или даже задонатить на кофе с сосиской в тесте.

4246410033771210

Кратко про экзамен:

- Первый (теоретический) вопрос - ОБЯЗАТЕЛЬНЫЙ. Если не ответить про Интернет, HTTP, состав запроса и ответа - дальше разговора нет.
- Оценка ассистента - это ваш максимум. Преподаватель спрашивает «вниз» от неё. Хорошая оценка ассистента очень важна.

Краткий гайд:

Если ответ на вопрос не понятен или вы считаете, что он недостаточен:

1. Копируете из этого файла весь вопрос полностью, вместе с ответом на него
 2. Открываете любую нейросеть
 3. Вставляете скопированное и дописываете единственную фразу "Объясни подробнее"
 4. В случае если все равно не понятно, то отправляем в чат нейросети следующую фразу "Объясни это мне так, будто я 5-ти летний ребенок"
- Если непонятно слово или фраза из ответа:

- Копируете непонятную вам часть
- Открывает гугл
- Вставляем в поисковую строку скопированное
- Дописываем ключевую фразу "С# что это"

Вопрос 1. Интернет: определение, принцип устройства, службы Интернет, понятие Интернет-сервера

Если не прочитали информацию выше(см. Предисловие), то вперед читать, оно обязательно!

Интернет - всемирная компьютерная сеть, построенная на основе стека протоколов TCP/IP.

Интернет состоит из 4 частей (это надо помнить обязательно):

- **Сеть TCP/IP** - сеть, построенная на стеке протоколов TCP/IP.
- **Службы Интернет** - набор служб (сервисов).
- **Организации** - которые поддерживают работоспособность и развитие Интернета.
- **Документация** - RFC и STD.

Протокол - правила обмена данными.

Службы (сервисы) Интернет

Служба - это две компоненты: сервер (программа, которая слушает сетевой порт) и протокол. Примеры служб: DNS, E-mail, FTP, Telnet, WWW.

Интернет-служба - сервер, который слушает порт от 1 до 65535 (0-1023 это системные порты, которые зарезервированы системой для основных протоколов, для прослушивания их нужны права администратора, порты 1024-49151 для пользовательских программ, остальные динамические, выдаются ОС приложениям в основном для исходящих подключений).

В ОС есть сокет - объект из двух компонентов: IP-адрес и порт (число от 1 до 2^{16}). Любая веб-программа является службой в Интернете.

Организации (для общего уровня - назвать несколько)

- **ISOC**(Internet Society) - владелец RFC, финансирует остальные организации.
- **IETF**(**I**nternet **E**ngineering **T**ask **F**orce (Инженерный совет Интернета)) - выпускает документацию RFC.
- **IAB**(Internet Architecture Board) - совет по архитектуре Интернета.
- **ICANN**(Internet Corporation for Assigned Names and Numbers) - управление доменными именами и IP-адресами.
- **IANA**(Internet Assigned Numbers Authority) - регистрация портов и типов MIME.
- **W3C**(World Wide Web Consortium) - веб-стандарты: HTTP, HTML, CSS, URI... Возглавляет Тим Бернерс-Ли.

Интернет-сервер (Интернет-ресурс)

Интернет-ресурс - сущность в сети Интернет, имеющая там свой адрес(будь то доменное или IP-адрес) (то, что опубликовано в Интернете).

Интернет-сервер - программа, которая прослушивает сетевой порт и обрабатывает приходящие на него запросы. Частный случай - веб-сервер (порт 80(HTTP) или 443(HTTPS)).

Акцент преподавателя: Курс называется «Технологии программирования в Интернет», поэтому определение Интернета и 4 его части спрашиваются обязательно. Определение - «моё», то есть преподавателя.

Вопрос 2. HTTP: определение, основные свойства, структура запроса и ответа, статусы, заголовки

HTTP - протокол прикладного уровня модели OSI для обмена данными между клиентом и сервером (правила обмена данными в вебе).

Основные свойства HTTP

- Есть версии: HTTP/1.1 (действующий, текстовый), HTTP/2 (бинарный), HTTP/3 (черновой).
- Два типа абонентов: клиент и сервер.
- Два типа сообщений: request (запрос) и response (ответ).
- От клиента к серверу идёт request, от сервера к клиенту - response.
- На один request всегда один response, иначе ошибка. На один response - один request, иначе ошибка.
- TCP-порты 80 и 443 закреплены за HTTP и HTTPS соответственно.
- Для адресации используется URI (стандарт STD 66).
- Поддерживается W3C, описан в нескольких RFC.

Структура запроса (Request)

- Метод (GET, POST, PUT, DELETE, HEAD, OPTIONS, TRACE, CONNECT) см. "HTTP Методы" в гугле.
- URI.
- Версия протокола (HTTP/1.1).
- Заголовки (пары имя-значение).
- Параметры (пары имя-значение).
- Расширение (тело, body).

Пример:

GET /path/resource?param1=value1&m2=value2 HTTP/1.1

(без тела и заголовков)

Полноценный пример:

POST /send-message.html HTTP/1.1

Host: mail.example.com

Referer: <http://mail.example.com/send-message.html>

User-Agent: BrowserForDummies/4.67b

Content-Type: multipart/form-data; boundary="Asrf456BGe4h"

Content-Length: *(суммарный объём, включая дочерние заголовки)*

Connection: keep-alive

Keep-Alive: 300

(пустая строка)

(отсутствующая преамбула)

--Asrf456BGe4h

Content-Disposition: form-data; name="DestAddress"

(пустая строка)

brutal-vasya@example.com

--Asrf456BGe4h

Content-Disposition: form-data; name="MessageTitle"

(пустая строка)

Я негодную

--Asrf456BGe4h

Content-Disposition: form-data; name="MessageText"

(пустая строка)

Привет, Василий! Твой ручной лев, которого ты оставил

у меня на прошлой неделе, разодрал весь мой диван.

Пожалуйста, забери его скорее!

Во вложении две фотки с последствиями.

--Asrf456BGe4h

Content-Disposition: form-data; name="AttachedFile1"; filename="horror-photo-1.jpg"

Content-Type: image/jpeg

(пустая строка)

(двоичное содержимое первой фотографии)

--Asrf456BGe4h

Content-Disposition: form-data; name="AttachedFile2"; filename="horror-photo-2.jpg"

Content-Type: image/jpeg

(пустая строка)

(двоичное содержимое второй фотографии)

--Asrf456BGe4h--

(отсутствующий эпилог)

Структура ответа (Response)

- Версия протокола.
- Код состояния (статус).
- Пояснение к коду состояния.
- Заголовки (пары имя-значение).
- Расширение (тело, body).

Пример ответа:

```
HTTP/1.1 200 OK
Date: Wed, 03 Jun 2026 01:45:00 GMT
Server: Apache/2.4.52 (Unix)
Content-Type: text/html;
charset=UTF-8
Content-Length: 125
Connection: keep-alive
<!DOCTYPE html>
<html>
<head>
<title>Example</title>
</head>
<body>
<h1>Привет, Мир!</h1>
</body>
</html>
```

Акцент преподавателя: В ответе параметров *НЕТ* - это любимый момент. Метод и параметры есть только в запросе.

Статусы ответа (5 серий)

- **1xx** - информационные сообщения.
- **2xx** - успешный ответ.
- **3xx** - переадресация.
- **4xx** - ошибка на стороне клиента.
- **5xx** - ошибка на стороне сервера.

Заголовки (привести по примеру каждого типа)

- **General** - общие, и в запросе, и в ответе (напр. Date, Connection).
- **Request** - только в запросе (напр. Accept, Host).
- **Response** - только в ответе (напр. Location, Server).
- **Entity** - описывают тело, и в запросе, и в ответе (напр. Content-Type, Content-Length).
- **X-Headers** - любые пользовательские заголовки, которые разработчик сам создал в ответе или запросе

Акцент преподавателя: *Есть пользовательские заголовки - их придумывает программист. Правило: имя должно начинаться с «X-» (например, X-My-Header). Об этом обязательно спрашивает.*

Вопрос 3. Web-приложение: определение, структура, общая схема

Web-приложение - клиент-серверное приложение, у которого клиент и сервер взаимодействуют между собой по протоколу HTTP. Точнее: клиент отправляет запросы по HTTP, а сервер отправляет ответы по HTTP.

Акцент преподавателя: *Это «очень простое определение, но половина не отвечает». Лучшая формулировка - именно «клиент отправляет запросы по HTTP, сервер отправляет ответы по HTTP».*

Клиент-серверное приложение состоит из 2 частей

- **Клиент** - инициатор связи, делает запрос (частный случай клиента - браузер).
- **Сервер** - обрабатывает запросы и отдаёт ответы.

От автора: Бывают случаи, например, когда наш сервер стучится(читай "обращается или отправляет запрос") на стороннее API(читай "к другому серверу"), он создает HTTP-запрос к нему, на той стороне стоит такой же сервер как и у нас, но делает он что-то другое. В таком случае клиентом будет уже наш сервер. В общем и целом клиент - все что может слать какие-либо запросы, не обязательно HTTP-запросы, а сервер - все, что может отправлять в ответ на запрос какой-либо ответ. Всякие WebHooks и WebSocket'ы не рассматриваем.

Структура и объекты веб-приложения (чем больше назовёте - тем лучше)

- Обработчики запросов.
- Конвейеры обработки данных (middleware).
- Фильтры.
- Объекты Request и Response.
- Cookie (куки).

- Сессия.
- Connection pool (пул соединений).
- Маршрутизация - про неё тоже надо рассказать.

Маршрутизация - механизм выбора обработчика HTTP-запроса. Основные критерии - HTTP-метод и URI.

Акцент преподавателя: Преподаватель просит: говоря про веб-приложение, обязательно упомянуть маршрутизацию и перечислить как можно больше объектов (cookie, сессия, request/response, connection pool).

Вопрос 4. ASP.NET CORE: назначение, свойства, кроссплатформенность, структура, маршрутизация, типы приложений, NuGet, встроенные компоненты

ASP.NET Core - программная платформа (кроссплатформенный фреймворк с открытым кодом) для разработки веб-приложений на платформе .NET.

Платформа - это язык программирования + библиотека + описание API + инструмент разработки.

Основные свойства (главное - кроссплатформенность)

Работает под Windows, Linux, Mac OS. В состав .NET входят: CoreCLR (исполняет IL-код(байт-код), CoreFX (базовая библиотека), языки C#, F#, VB.NET, менеджер пакетов NuGet. CoreCLR и CoreFX - разные под разные ОС(нативная реализация системных вызовов, что обеспечивает различия в реализации под каждую ОС).

Кроссплатформенность - возможность запускать программу на разных операционных системах.

Акцент преподавателя: Обязательно спросит: какие бывают кроссплатформенности? Ответ - на уровне исходного кода и на уровне байт-кода. У ASP.NET Core - на уровне байт-кода (IL): программу создаём, например, на Windows, а используем на Mac OS благодаря разным CLR и нативным библиотекам.

Общая структура приложения (любое ASP.NET Core состоит из):

- **Host (хост)** - программа-процесс ОС, образует исполняемый файл; внутри неё - логика приложения.
- **HTTP-сервер (Kestrel / OWIN Server)** - принимает запрос(так же управляет ими) и создаёт объект HttpContext.
- **Middleware** - двунаправленный конвейер обработки запросов и ответов.
- **Application** - обработчик-конечная точка конвейера.
- **Services** - сервисы, какая-то логика которую можно инжектировать внутрь обработчика (Dependency Injection).

Маршрутизация - механизм выбора обработчика HTTP-запроса. Специальный middleware-элемент по методу и URI находит конечную точку. Если точка не найдена - ответ 404(Not found).

Типы приложений ASP.NET Core (которые мы рассматриваем)

- Minimal API (мини API) (CSR - Client Side Rendering, когда наш сервер отдает только данные а рендеринг происходит либо на клиенте(в браузере например) либо на каком-то другом сервере, одним словом кто-угодно, но не мы).
- Web Page (Razor Page) (SSR - Server Side Rendering, когда наш сервер отвечает за непосредственный рендер итоговой страницы, когда сервер сам строит например HTML разметку, сам заполняет её данными и отдает клиенту готовую полностью разметку).
- MVC.

Акцент преподавателя: *Web API мы НЕ трогаем (вынесено на следующий семестр). Назвать достаточно эти три.*

Менеджер NuGet

NuGet - программный менеджер пакетов; пакет содержит одну или несколько библиотек. Есть централизованное хранилище. Пример: `dotnet add package Newtonsoft.Json`. Все добавленные пакеты видны в файле проекта.

Встроенные компоненты

- Структура middleware (обработчики).
- Встроенный Dependency Injection.
- Маршрутизация.
- Конфигурирование и журналирование.
- HTTP-сервер Kestrel.

Вопрос 5. ASP.NET CORE: проект, файлы (launchSettings.json, appsettings.json, Program.cs), builder, HttpContext, wwwroot

Назначение файлов проекта

- **.sln** - решение.
- **.csproj** - как собрать проект или же другими словами конфигурационный файл сборки; в нём видны подключённые NuGet-пакеты и другие проекты необходимые в сборке.
- **launchSettings.json** - как запустить (run) проект: URL, переменные окружения, профили запуска. Нужен только для разработки в Visual Studio.

- **appsettings.json** - конфигурация в режиме выполнения(они же runtime настройки/конфиги).
- **appsettings.Development.json** - конфигурация в режиме Development.
- **Program.cs** - точка входа: builder, конечные точки, middleware, маршрутизация, журналирование.

Акцент преподавателя: Любимые вопросы: 1) При деплое файл `launchSettings.json` НЕ переносится (он только для запуска из Visual Studio). 2) В результате компиляции получается DLL (exe не нужен, за ним стоит DLL)(от автора: В .NET .exe файл является лишь небольшим загрузчиком который в свою очередь обращается к DLL файликам которые мы имеем после сборки, при этом те же механизмы загрузки есть и в самом dotnet CLI утилите(та что вызывается когда пишем в консоли dotnet).

Builder и основные настраиваемые компоненты

Приложение строится по паттерну builder. Сначала создаём builder (`WebApplicationBuilder`), настраиваем его, затем методом `Build()` получаем объект `WebApplication` (это и есть приложение, ядро с сервером Kestrel), запускаем методом `Run()`.

- `builder.Services` - регистрация сервисов (DI).
- `builder.Configuration` - конфигурация.
- Журналирование (logging).
- Настройка Kestrel.

```
var builder = WebApplication.CreateBuilder(args);
// builder.Services.Add... // настройка
WebApplication app = builder.Build();
app.MapGet("/", () => "Hello");
app.Run();
```

HttpContext - объект, создаваемый HTTP-сервером (Kestrel) из битовой последовательности запроса; содержит исчерпывающую информацию о соединении, запросе и будущем ответе (в т.ч. `Request` и `Response`).

`Response` в начале - пустая «болванка», она заполняется в `Application` в зависимости от запроса, а потом отправляется клиенту.(от автора: при том, респонс не может вернуться абсолютно пустым, там всегда должен быть статус код, но так же бывают случаи когда например мы шлем запрос на порт или адрес, который этого не ожидает, тогда подключение сбрасывается на уровне сети, а клиент сам обрабатывает такую ситуацию, так же и в случае `timeout` когда ответ от сервера не пришел за выделенное время, как такового ответа от сервера физически нет, но клиент сам определяет такую ситуацию и формирует для себя ответ т.к. как мы помним на реквест всегда будет респонс, просто респонс не всегда формируется сервером)

wwwroot - специальная директория для статических файлов(статика, они же статические файлы, это файлы которые с течением времени не меняются и лежат на сервере неизменными пример: картинки, текстовые файлы и др.); именно отсюда по умолчанию отдаются статические ресурсы клиенту.

Вопрос 6. ASP.NET CORE: middleware - определение, назначение, принцип устройства, встроенные элементы, разработка своего элемента

Middleware - двунаправленный конвейер обработки запросов и ответов; реализация паттерна «цепочка ответственности» (chain of responsibility).

Принцип устройства

HttpContext движется по конвейеру: веб-сервер (Kestrel) → элементы middleware → конечная точка (Application), а затем в обратном порядке. Это вложенные вызовы: каждый предыдущий элемент вызывает следующий через делегат next, а потом управление возвращается обратно.

Акцент преподавателя: *Не забыть: это ВЛОЖЕННЫЕ вызовы. До next мы обрабатываем запрос, после next - ответ.*

Каждый элемент может: получить request и сгенерировать response (не пускать дальше, в таком случае конвейер разворачивается именно в этом месте); изменить request и передать дальше; изменить исходящий response и передать назад.

Принцип разработки своего (пользовательского) элемента

- Обработчик принимает 2 параметра: HttpContext (context) и делегат next.
- До вызова next - обрабатываем запрос.
- Вызов await next(context) - переход на следующий элемент.
- После next - обрабатываем ответ.
- Порядок подключения важен: если перепутать, приложение может не работать.

```
app.Use(async (context, next) =>
{
    // обработка запроса
    await next(context); // переход к следующему
    // обработка ответа
});
```

Встроенные компоненты, подключаемые как middleware

- UseStaticFiles - статические файлы.
- UseRouting / EndpointMiddleware - маршрутизация.
- UseExceptionHandler / DeveloperExceptionPage - обработка ошибок.

- UseAuthentication, UseAuthorization - экранирование эндпоинтов аутентификацией/авторизацией на основе зашифрованных cookie файлов отданных клиенту ранее
- UseHttpLogging - журналирование.

Вопрос 7. ASP.NET CORE: статические файлы - назначение, применение, скачивание

Статический файл - файл, который находится на стороне сервера и без изменения может быть переслан на сторону клиента.

Примеры: HTML, CSS, JavaScript, картинки, видео и тд. - это всё статические файлы.

Применение

По умолчанию статические файлы лежат в папке wwwroot. Чтобы сервер их отдавал, подключают middleware-элемент UseStaticFiles. Есть и UseDefaultFiles - тогда index.html / default.html ищутся автоматически.

```
app.UseStaticFiles();
app.UseDefaultFiles();
```

Как клиент получает статический файл (любимый вопрос «покажи в HTML»)

В HTML-коде теги `<link>`, `<script>`, `<img>` приводят к тому, что браузер генерирует отдельный GET-запрос за этим статическим ресурсом по указанному в теге адресу. Например:

<code><link rel="stylesheet" href="/site.css"></code>	<code><!-- GET /site.css --></code>
<code><script src="/app.js"></script></code>	<code><!-- GET /app.js --></code>
<code></code>	<code><!-- GET /photo.jpg --></code>

Акцент преподавателя: И `<link>`, и `<script>`, и `<img>` приводят к GET-запросу за статическим ресурсом. На сервере за выдачу отвечает wwwroot + UseStaticFiles.

Вопрос 8. Minimal API: структура, конечные точки MapXXX, UseExceptionHandler, Fallback, обработка параметров, Content-Type / Accept, MapGroup

Minimal API - приложение, которое содержит простейшие конечные точки с простейшей маршрутизацией. Маршрутизация осуществляется по методу и по URI.

Конечная точка (endpoint) - HTTP-обработчик, выполняющий обработку запроса и возвращающий ответ.

Конечные точки MapXXX

Регистрируются методами MapGet, MapPost, MapPut, MapDelete и тд. Первое слово в названии метода всегда Map(карта, аналогия с картой городов, где города - конечные

точки а.к.а. эндпоинты), а второе слово - название HTTP метода. Первый аргумент - шаблон URI, второй - функция-обработчик.

```
app.MapGet("/", () => "Hello World");
app.MapGet("/product/{id}", (int id) => $"product {id}");
app.MapPost("/product", (Product p) => Results.Ok(p));
```

UseExceptionHandler и Fallback-обработчик

- **UseExceptionHandler("/error")** - middleware-обработчик ошибок; ставится в самом верху конвейера. При исключении управление переходит на указанную точку.
- **Fallback-обработчик (MapFallback)** - «запасная» точка: срабатывает, если ни одна другая конечная точка не подошла к запросу.

Обработка параметров и входных данных

Параметры конечной точки заполняются привязкой модели по именам: из пути (path), из строки запроса (query, параметры после знака «?», они же query-параметры), из заголовков, из тела запроса body (JSON).

Заголовки Content-Type и Асцепт

- **Content-Type** - указывает формат тела сообщения (например application/json) - какой формат фактически отправлен. Обязателен в POST, PUT и PATCH запросах, т.к. показывает какой тип body передан, иначе сервер не поймет как обрабатывать тело запроса. В ответах он заполняется автоматически в зависимости от содержимого HTTP-сервером, формально не обязателен в ответе, но если его явно УДАЛИТЬ из ответа, то браузер попытается его угадать, что небезопасно и в целом ай-ай-ай.

Пример:

Content-Type: application/json

Content-Type: multipart/form-data

- **Асцепт** - клиент сообщает, в каком формате он хочет(читай «может») получить ответ.

Пример:

Асцепт: application/json

Асцепт: text/html

Маршрутизация с помощью MapGroup

MapGroup задаёт общий префикс для группы конечных точек, чтобы не повторять его в каждой.

```
var g = app.MapGroup("/api");
g.MapGet("/users", () => ...); // /api/users
g.MapGet("/cars", () => ...); // /api/cars
```

Вопрос 9. Minimal API: фильтры конечных точек, обработка параметров (GetArgument), делегат next

Фильтр конечной точки - обработчик, который встраивается вокруг конечной точки и работает и ДО, и ПОСЛЕ неё.

Принцип работы (как у middleware)

- Фильтр получает контекст `EndpointFilterInvocationContext`.
- Через него методом `GetArgument` можно получить аргументы конечной точки.
- Есть делегат `next`, с помощью которого переходим от фильтра к фильтру (и к самой точке).
- До `next` - обрабатываем запрос, после `next` - обрабатываем ответ.

```
app.MapGet("/p/{id}", (int id) => id)
.AddEndpointFilter(async (context, next) =>
{
    var id = context.GetArgument<int>(0); // до next: запрос
    var result = await next(context); // вызов точки
    return result; // после next: ответ
});
```

Акцент преподавателя: Главное сказать: фильтры работают до и после точки; `GetArgument` даёт аргументы; `next` - переход между фильтрами.

Вопрос 10. Minimal API: скачивание файлов

Чтобы отдать (скачать) файл клиенту, конечная точка возвращает результат `Results.File` - указывается путь к файлу (или поток/массив байт) и тип содержимого (`Content-Type`).

```
app.MapGet("/photo/{name}", (string name) =>
{
    string path = Path.Combine(photoDir, name);
    return Results.File(path, "image/jpeg");
});
```

В лабораторной с «Celebrities» так отдавались фотографии: конечная точка по имени файла из настроенной папки возвращала картинку через `Results.File`.

Акцент преподавателя: Преподаватель может попросить открыть лабораторную и показать, где идёт скачивание файла, или дописать кусочек кода. Лабораторные приносить с собой обязательно.

Вопрос 11. Minimal API: шаблоны и констрейнты маршрутов

Шаблон маршрута - символическое выражение, задающее множество URI. URI соответствует шаблону, если он принадлежит этому множеству. Например, шаблон `{id}` задаёт `/1`, `/10`, `/abc` и т.д.

Констрейнт (constraint) - ограничение на параметр маршрута: задаёт, какие значения допустимы (тип, диапазон, длина).

Примеры стандартных констрейнтов

- `{qty:int}` - любая строка, преобразуемая в целое.
- `{id:guid}` - глобально уникальный идентификатор.
- `{cost:decimal}` - значение `decimal`.
- `{age:min(18)}` - число не меньше 18.
- `{qty:int?}` - необязательный целочисленный параметр.
- `{name:maxlength(20)}` - ограничение длины строки.

```
app.MapGet("/product/{id:int}", (int id) => ...);
app.MapGet("/user/{key:guid}", (Guid key) => ...);
```

Вопрос 12. Minimal API: LinkGenerator, поименованные точки (WithName), переадресация RedirectToRoute

Конечной точке можно дать имя методом `WithName()`. По имени сервис `LinkGenerator` умеет сгенерировать ссылку (URI) на эту точку (`GetPathByName`). `RedirectToRoute` делает переадресацию на поименованную точку.

Акцент преподавателя: Сам по себе `LinkGenerator` преподаватель из вопросов убрал как неважный. А вот процедуру переадресации (`Redirect`) надо знать **НАИЗУСТЬ** - это главное в этом вопросе.

Переадресация (Redirect) - процедура, когда сервер не может или не хочет обрабатывать запрос, но знает, на какой конечной точке его можно обработать. Тогда сервер генерирует ответ с кодом 3xx и заголовком `Location`, в котором указывает новый URI. Клиент видит код 3xx и делает новый GET запрос по URI из заголовка `Location`.

Детали по кодам (важно)

Все коды начинающиеся на цифру 3 означают редиректы.

Таблица кодов редиректов

300 Multiple Choices	У запроса более одного возможного ответа. Пользовательский агент или пользователь должен выбрать один из них. Не существует стандартизированного способа выбора одного из ответов, но рекомендуется использовать HTML-ссылки на возможные варианты, чтобы у пользователя была возможность выбора.
301 Moved Permanently	URL-адрес запрошенного ресурса был изменен навсегда. Новый URL-адрес указан в ответе.
302 Found	URI запрошенного ресурса был временно изменен. В будущем могут быть внесены дальнейшие изменения в URI. Следовательно, этот же URI должен использоваться клиентом в будущих запросах.
303 See Other	Клиенту необходимо получить запрошенный ресурс по другому URI с помощью запроса GET.
304 Not Modified	Этот код используется для целей кэширования. Он сообщает клиенту, что ответ не был изменен, поэтому клиент может продолжать использовать кэшированную версию ответа.
307 Temporary Redirect	Клиенту необходимо получить запрошенный ресурс по другому URI тем же методом, который использовался в предыдущем запросе. Он имеет ту же семантику, что и код ответа 302 Found, за исключением того, что пользовательский агент не должен изменять используемый метод: если в первом запросе использовался POST, то POST должен использоваться и во втором запросе.
308 Permanent Redirect	Ресурс теперь находится по другому URI, указанному в заголовке ответа Location. Он имеет ту же семантику, что и код ответа 301 Moved Permanently, за исключением того, что пользовательский агент не должен изменять используемый метод: если в первом запросе использовался POST, то POST должен использоваться и во втором запросе.

- **300-306** - клиент делает новый запрос методом GET по новому URI.
- **307 и 308** - клиент делает тот же запрос с тем же методом, сохраняя тело и все параметры - просто по новому адресу.

Акцент преподавателя: *Вся процедура переадресации описана в протоколе HTTP. Если клиент и сервер поддерживают HTTP, они обязаны так себя вести.*

Вопрос 13. Minimal API: привязка модели (Model binding), атрибуты FromXXX, собственные типы, необязательные параметры, массивы

Привязка модели (Model binding) - механизм, который ставит в соответствие параметры, пришедшие в запросе, параметрам вызываемого метода (конечной точки). Иначе - преобразование битовой последовательности запроса в POCO-объект.

POCO (Plain Old CLR/Class Object) - это простой независимый класс (объект), который содержит только данные (поля и свойства) и служит для хранения информации.

Привязка идёт по именам: имя в запросе должно совпадать с именем параметра. Осуществляется на этапе EndpointMiddleware. Привязывать можно данные из: path, query, headers, JSON-body, DI-объекты.

Акцент преподавателя: *Minimal API НЕ умеют автоматически привязывать данные с mime-типом application/x-www-form-urlencoded (HTML-форма). Тогда используют HttpRequest.Form.*

Атрибуты-подсказки FromXXX (откуда брать параметр)

- [FromRoute] - из пути.
- [FromQuery] - из строки запроса.
- [FromHeader] - из заголовка.
- [FromBody] - из тела (JSON).
- [FromServices] - из сервисов (DI).

Собственные типы в шаблоне маршрута

Чтобы использовать свой тип как параметр маршрута, в этом типе надо реализовать метод TryParse. Он проверяет строку из URI: если получилось распарсить - маршрутизатор выбирает эту точку, если нет - обходит её.

```
public static bool TryParse(string s, out MyType result) { ... }
```

Необязательные параметры и массивы

- Необязательный параметр - со знаком «?» или со значением по умолчанию.
- Массивы передаются в query-параметрах, можно указать [FromQuery(Name="...")].

Вопрос 14. Minimal API: хорошо известные типы в параметрах

Хорошо известные типы - ряд типов, которые в Minimal API могут быть автоматически внедрены и указаны в качестве параметров конечной точки - они распознаются платформой без всяких атрибутов.

Примеры

- HttpContext
- HttpRequest
- HttpResponse
- CancellationToken

- ClaimsPrincipal (пользователь)
 - ILogger / ILogger<T> - используется для логирования внутри конечной точки.
- LinkGenerator - для динамического создания URL-адресов на другие эндпоинты приложения.
- IFormFileCollection и IFormFile - для работы с загружаемыми файлами (мультипарт-запросы).
- Stream или PipeReader - для чтения тела запроса напрямую в виде потока данных.
- TextReader - аналогично, для чтения тела запроса как текстового потока.

Как Minimal API отличает эти типы от обычных параметров? (Механизм работы)

Minimal API использует строгие правила для определения того, откуда брать параметр:

Если тип параметра находится в списке «хорошо известных» (как HttpContext), платформа берет его напрямую из контекста текущего запроса.

- Если тип зарегистрирован в DI-контейнере (Dependency Injection), он внедряется как служба.
- Если это простой тип (строка, число, GUID) или тип с методом TryParse, платформа пытается привязать его из маршрута (Route), строки запроса (Query) или заголовков (Headers).
- Если это сложный объект (класс), платформа по умолчанию считает, что он должен прийти из тела запроса (Body) в формате JSON.

3. Нюансы и правила привязки

Службы DI без атрибутов: Начиная с .NET 7, если тип зарегистрирован в DI-контейнере, вам больше не нужно писать атрибут [FromServices]. Платформа сама поймет, что это зависимость, если только это не «хорошо известный» тип или не простой тип данных.

Явное указание атрибутов: Если «хорошо известный» тип (например, ClaimsPrincipal) нужно по какой-то причине получить не из контекста, а из DI или другого места, автоматическое поведение можно переопределить с помощью атрибутов (например, [FromServices]).

```
app.MapGet("/info", (HttpContext ctx, HttpRequest req) =>
{
    return req.Path; // типы распознаются автоматически
});
```

Вопрос 15. Minimal API: пользовательские привязки, атрибут AsParameters

Пользовательская привязка - собственный механизм привязки, когда стандартной не хватает: в типе реализуется статический метод BindAsync, который

сам разбирает запрос и создаёт объект.

В Minimal API привязка моделей (Model Binding) по умолчанию работает автоматически, разделяя параметры на простые типы (из URL/Query) и сложные типы (из тела запроса в формате JSON). Однако для кастомной логики и улучшения архитектуры кода платформа предоставляет два мощных инструмента: **пользовательские привязки** и атрибут **[AsParameters]**.

Атрибут [AsParameters] (появился в .NET 7) реализует паттерн Parameter Object (Объект параметров). Он предназначен для решения проблемы «раздутых сигнатур» (Clean Code).

Какую проблему он решает?

Если конечной точке требуется много параметров (например: ID из маршрута, параметры пагинации из query, логгер из DI и токен отмены), то сигнатура метода становится огромной и нечитаемой. [AsParameters] позволяет объединить все эти параметры в единую структуру или класс.

Уникальные возможности [AsParameters]:

Внутри структуры/класса можно комбинировать абсолютно любые источники данных:

- Атрибуты привязки: [FromRoute], [FromQuery], [FromHeader], [FromBody].
- Автоматически внедряемые сервисы из DI-контейнера.
- «Хорошо известные типы» (например, CancellationToken или HttpContext).

```
public static ValueTask<MyType?> BindAsync(HttpContext ctx) { ... }
```

[AsParameters] - атрибут, который применяется к структуре или классу и позволяет привязать сразу все параметры к свойствам этого класса. Внутри класса для свойств можно применять атрибуты FromXXX.

```
struct Args
{
    [FromQuery] public int Page;
    [FromRoute] public int Id;
}
app.MapGet("/p/{id}", ([AsParameters] Args a) => ...);
```

Пользовательские привязки (Custom Binding)

Если стандартная привязка не справляется с задачей, разработчик может определить собственную логику парсинга HTTP-запроса. Для этого внутри целевого типа данных реализуется один из двух специальных статических методов. Платформа распознает их автоматически с помощью механизмов генерации кода (Expression Trees) при старте приложения.

А. Метод TryParse (для простых преобразований)

Используется, когда нужно преобразовать одну строку (из Query, Route или Headers) в сложный объект.

Сигнатура: `public static bool TryParse(string? value, IFormatProvider? provider, out T result)`

Пример применения: Парсинг строки 55.75,37.61 из Query-параметра в структуру Point(X, Y).

Б. Метод BindAsync (для глубокой работы с контекстом)

Используется, когда для создания объекта требуется доступ ко всему HTTP-контексту (заголовки, куки, сессия или тело запроса).

Сигнатура: `public static ValueTask<T> BindAsync(HttpContext context)`

Пример реализации BindAsync:

Представим, что нам нужно автоматически извлекать кастомный токен безопасности из заголовков запроса:

```
public class SecurityToken
{
    public string TokenValue { get; set; } = string.Empty;

    // Метод автоматически вызывается платформой перед выполнением
    // эндпоинта
    public static ValueTask<SecurityToken?> BindAsync(HttpContext context)
    {
        var token = context.Request.Headers["X-Auth-Token"].FirstOrDefault();

        if (string.IsNullOrEmpty(token))
        {
            return ValueTask.FromResult<SecurityToken?>(null); // Возвращаем null, если заголовок пуст
        }

        var result = new SecurityToken { TokenValue = token };

        return ValueTask.FromResult<SecurityToken?>(result);
    }
}
```

```
// Использование в эндпоинте (чисто и без лишних атрибутов):
app.MapGet("/secure-data", (SecurityToken? token) =>
{

    if (token == null) return Results.Unauthorized();

    return Results.Ok($"Доступ разрешен для токена: {token.TokenValue}");

});
```

Вопрос 16. Minimal API: валидация параметров запроса встроенным механизмом (MinimalApis.Extensions)

Валидация - проверка пришедших данных на корректность.

Рассказывать надо принципы. Речь о валидации данных, пришедших в теле POST-запроса или другого метода, но с телом, например PUT, PATCH (проверяется корректность JSON и значений).

Принцип встроенной валидации

- У нас есть модель данных - это класс.
- Для каждого поля в модели прописываются специальные атрибуты валидации.
- Примеры атрибутов: Required, Min, Max, MaxLength, проверка на e-mail, на телефон, на регулярное выражение.
- Этот встроенный механизм называется MinimalApis.Extensions (подключается расширением, напр. WithParameterValidation).

```
class Product
{
    [Required] public string Name { get; set; }

    [Range(1,1000)] public int Price { get; set; }
}

app.MapPost("/p", (Product p) => ...).WithParameterValidation();
```

Акцент преподавателя: Это первый, встроенный подход (атрибуты в классе). Есть и второй - Fluent Validation (отдельный фреймворк, проверки выносятся в отдельный класс), но он на следующий семестр. Сообщения об ошибках регламентируются RFC 7807 / 9457.

Вопрос 17. ASP.NET CORE: принцип разработки middleware-обработчика ошибок (Use, UseMiddleware, IApplicationBuilder)

Обработчик ошибок - это обычный middleware-элемент, который ставят в самом верху конвейера (в начале). Разрабатывается так же, как любой middleware.

Главное - принцип распространения исключения в C#

- Если на глубине стека вызовов возникает exception и нет обработчика, исключение поднимается вверх по стеку вызовов пока не найдется обработчик, который его корректно словит(catch) В случае, если так и не найдется обработчик, то клиенту вернется ответ с кодом 500 Internal Server Error и детали ошибки отобразятся в логах сервера.
- Оно ищет подходящий обработчик.
- Если мы поставили свой обработчик в начале middleware-конвейера, исключение в конце концов докатится до него (если его раньше нигде не перехватили).

Способы подключения

- **app.Use(...)** - встроить обработку прямо лямбдой (context, next).
- **app.UseMiddleware<T>()** - подключить свой класс-middleware.
- **Расширение IApplicationBuilder** - свой метод-расширение UseXxx для красивого подключения.

```
app.UseMiddleware<ExceptionHandlerMiddleware>();  
// или  
app.Use(async (ctx, next) => {  
    try { await next(ctx); }  
    catch (Exception e) { /* сформировать ответ об ошибке */ }  
});
```

Вопрос 18. ASP.NET CORE: Dependency Injection - понятие, Transient / Singleton / Scoped, FromServices

Сервис - класс (объект класса) с заданным жизненным циклом, который создаётся в автоматическом (полуавтоматическом) режиме; в нём описывается бизнес-логика нашего приложения.

Dependency Injection (DI) - механизм, позволяющий получать объект сервиса в автоматическом режиме (через параметры) с заданным жизненным циклом. DI - частный случай подхода IoC (инверсия управления).

Идея: вынести зависимости за пределы кода, чтобы получить слабосвязанный код. Логику можно поменять, а приложение не пострадает.

Два способа получить сервис

- **Service Locator** - сами «вытаскиваем» сервис: `GetService<T>`, `GetRequiredService<T>` (пример - `LinkGenerator`).

- **Dependency Injection** - платформа сама передаёт сервис как параметр метода/конструктора.

Жизненный цикл сервисов

- **Transient** - создаётся при каждом обращении (каждый раз, когда пишем например `GetRequiredService<S>` будет создаваться новый экземпляр).
- **Scoped** - в рамках одного запроса (scope, другими словами внутри функции обработчика запроса) ведёт себя как Singleton(т.е. когда мы в первый раз вызываем в обработчике `GetRequiredService<S>` то создастся новый экземпляр, а когда потом в этом же обработчике(читай «функции») вызовем `GetRequiredService<S>`, то будет использоваться тот же экземпляр, что создался при первом вызове), а в разных запросах - как Transient (на каждый запрос свой).
- **Singleton** - создаётся один раз и всегда возвращается один и тот же объект(от запуска сервера, до его выключения будет все время один и тот же объект(читай «экземпляр класса»).

```
builder.Services.AddTransient<IRepo, Repo>();  
builder.Services.AddScoped<IRepo, Repo>();  
builder.Services.AddSingleton<IRepo, Repo>();
```

От автора: По поводу DI - это очень значимый подход в современной разработке. Без него тяжело придумать любое API, это подход, который позволяет легко менять, тестировать и в целом поддерживать код. Когда проект вмещает в себя 2000 строчек кода, кажется, что не такая уж и проблема в случае чего переписать не 10 строчек, а 30, но если(читай «когда») проект разрастется до сотен тысяч строк и тысяч файлов и классов, то можно крышей поехать это все править, ни одна нейронка не справится. Отсюда и такая важность. В целом подход заключается в вынесении какой-либо логики в отдельный слой(читай «сервис»), который вы инжектируете, когда вам это надо, а остальное за вас делает фреймворк(что такое фреймворк?).

Акцент преподавателя: Вопрос «на 9-10»: как отличить, что сервис transient, а что scoped, если заранее не знаешь? Ответ: в рамках ОДНОГО запроса запросить сервис дважды. Если оба раза один и тот же объект - это Scoped; если каждый раз новый - Transient. [FromServices] подсказывает взять параметр из сервисов.

Вопрос 19. Razor Page: паттерн MVVM, структура (View, Model), Program.cs, маршрутизация, обработка запроса, DI

Razor Page - компонент в составе ASP.NET Core, основная задача которого - обрабатывать cshtml-файлы. Это многостраничное приложение с рендерингом на сервере. См. вопрос №4

MVVM (Model-View-ViewModel) - паттерн, по которому устроена Razor Page: представление (View) - cshtml-разметка, а модель (PageModel) хранит данные и обработчики для страниц; публичные свойства модели реализованной в PageModel доступны в разметке (View а.к.а. cshtml-файл).

Структура (View + Model)

- **View** - файл .cshtml - разметка страницы.
- **Model (PageModel)** - файл .cshtml.cs - данные и обработчики запросов (OnGet, OnPost и тд.).

Program.cs

```
builder.Services.AddRazorPages();  
var app = builder.Build();  
app.MapRazorPages();
```

Маршрутизация и порядок обработки запроса

- Маршрут к странице соответствует её пути в папке Pages (можно переопределить в @page).
- GET-запрос обрабатывает метод OnGet, POST - OnPost.
- Обработчик заполняет свойства модели, затем Razor генерирует разметку в body ответа.

Применение DI в обработчиках

Сервисы внедряются в PageModel через конструктор (или через @inject прямо в разметку).

Вопрос 20. Razor Page: Razor-разметка @{...}, @page, @using, @inject, @model, @functions, @if, @foreach

Директивы обозначаются символом @. Это код Razor, написанный в синтаксисе C# и компилируемый Razor Engine.

Что можно написать внутри (преподаватель просит перечислить всё)

- **@{ ... }** - блок кода C#.
- **@page** - говорит, что это страница; здесь же можно указать маршрут.
- **@using** - то же назначение, что и using в коде - сокращение имён namespace.
- **@inject** - внедрение зависимости (DI) в представление (View). Обычно используется для инъекции узконаправленных сервисов для UI.
- **@model** - задаёт тип модели - тип параметра, который передаётся при вызове представления. Не путать с @Model - это нужно для обращения к модели в разметке

- **@functions** - объявление функций, работающих в рамках этого представления.
- **@if, @foreach, @for** - операторы в синтаксисе C#; на самом деле это не операторы C#, а конструкции, компилируемые Razor, но в целом это работает так же как в C#.

Пример:

```
@page
@model IndexModel
@using MyApp.Data
@inject IRepo Repo
@{ var n = 5; }
@foreach (var x in Model.Items) { <li>@x.Name</li> }
```

Вопрос 21. Razor Page: Layout-макетирование, Layout.cshtml.css, директивы @RenderBody, @RenderSection, @section

Макет (Layout) - такой же cshtml-файл, который может быть применён к макетируемому cshtml-файлу (общий каркас страниц - шапка, меню, подвал).

Нейрослоп для понимания: **Layout (макет)** — это «трафарет» или общая HTML-рамка для страниц сайта.

Представь, что у тебя на сайте 50 страниц. У всех у них абсолютно одинаковая шапка (Header), боковое меню (Sidebar) и подвал (Footer). Отличается только контент по центру. Вместо того чтобы копировать тысячи строк HTML-кода из файла в файл, ты выносишь всю общую разметку в один файл-макет (`_Layout.cshtml`), а в изменяемых страницах пишешь **только уникальный контент**.

Во время запроса к серверу Razor Engine берет твою страницу, берет макет, «склеивает» их в один документ и отправляет пользователю.

Как применяется

- Чтобы стать макетируемым, cshtml-файл указывает переменную `Layout = "имя макета"`.
- Тогда этот макет применяется к данной странице.

Директивы макетирования

- **@RenderBody()** - место в макете, куда вставляется всё основное содержимое макетируемой страницы (то, что не попало в секции).
- **@RenderSection("имя", false)** - место для именованной секции; второй параметр `false` - секция необязательная (`true` - обязательная).

- **@section имя { ... }** - в макетируемой странице задаёт содержимое, которое попадёт в соответствующий `RenderSection` в макете.

Файл `Layout.cshtml.css` - это стили, относящиеся именно к данному макету (изолированные CSS).

Если рядом с файлом `_Layout.cshtml` создать файл `_Layout.cshtml.css`, включится механизм **CSS Isolation**:

1. Стили, написанные в этом файле, применяются **только к элементам самого макета** (к его тегам `header`, `footer` и т.д.).
2. Они **не повлияют** на элементы, которые рендерятся внутри `@RenderBody()` или `@RenderSection`.
3. **Как это работает технически:** При компиляции Razor автоматически добавляет уникальный атрибут (например, `b-398gfhf763`) к HTML-тегам макета и к CSS-селекторам в файле стилей. В итоге селектор `h1` превращается в `h1[b-398gfhf763]`. Это защищает глобальные стили приложения от случайных конфликтов и «расползания» верстки.

Пример макетирования:

Shared/_Layout.cshtml

```
<!DOCTYPE html>
<html>
<head>
  <title>Мой Сайт</title>
  <link rel="stylesheet" href="~/css/site.css" />
</head>
<body>
  <header>Шапка сайта (Логотип, Меню)</header>

  <main>
    @RenderBody()
  </main>

  <footer>Подвал сайта (2026 год)</footer>

  @RenderSection("Scripts", required: false)
</body>
</html>
```

Pages/Contacts.cshtml

```
@page
@{
  Layout = "_Layout";
}
```

```
<h1>Наши контакты</h1>
<p>Телефон: +375 (XX) XXX-XX-XX</p>
@section Scripts {
    <script src="~/js/contacts-map-plugin.js"></script>
}
```

Вопрос 22. Razor Page: привязка параметров к модели (mapping)

Привязка (mapping) ставит в соответствие данные запроса свойствам/параметрам модели. В Razor Page это делается несколькими способами:

- **Атрибут [BindProperty]** - свойство модели автоматически заполняется из запроса (часто из формы при POST-запросах).
- **Параметры методов** - значения можно принимать прямо параметрами OnGet/OnPost.
- **Атрибуты FromXXX** - [FromQuery], [FromRoute] - подсказывают, откуда брать значение.

```
[BindProperty] public Product Item { get; set; }
public void OnGet([FromQuery] int page) { ... }
```

Акцент преподавателя: Ничего сложного: главное сказать, какими способами и откуда осуществляется привязка (bind, параметры методов, атрибуты).

Вопрос 23. Razor Page: работа с формами, обработка параметров формы, upload файлов с клиента на сервер

Форма - HTML-тег `<form>` с полями ввода; служит для отправки данных с клиента на сервер.

Схема работы с формой

- Сначала GET-запрос - сервер присылает разметку с формой и полями.
- Пользователь заполняет поля и отправляет форму - обычно POST-запросом или другим в зависимости от цели формы(что делает и для чего).
- Параметры формы передаются в теле запроса. Имя параметра берётся из атрибута name тега input, значение - из value.
- Razor engine все равно на регистр, в котором написано имя, но значение чувствительно к регистру.

Поиск параметров идет в следующей порядке:

- Значения формы (`Form data` , переданные в теле POST-запроса).

- Значения маршрута (`Route data` , например, `/Pages/Product/5`).
- Строка запроса (`Query string` в URL, например, `?age=20`).

Акцент преподавателя: После успешной обработки формы часто делают переадресацию (*Redirect*) на другую страницу - паттерн *PRG (Post-Redirect-Get)*.

Про PRG:

Паттерн разделяет отправку данных и отображение результата:

1. **POST:** Пользователь отправляет заполненную форму. Сервер сохраняет данные в БД.
2. **REDIRECT:** Вместо отправки HTML-страницы сервер возвращает HTTP-статус `302 Found` (или `303 See Other`) с заголовком `Location`, указывающим на адрес другой страницы (например, страницы успешного завершения или списка товаров).
3. **GET:** Браузер автоматически делает безопасный `GET`-запрос на указанный адрес и отображает страницу. Теперь обновление по F5 просто перезагрузит эту финальную `GET`-страницу, не отправляя данные формы заново.

В Razor Pages это реализуется методами обработчиков (`OnPost`):

```
public IActionResult OnPost()
{
    if (!ModelState.IsValid) return Page(); // Возврат той же страницы (не PRG, т.к. есть ошибки)

    // ... сохранение в базу данных ...

    return RedirectToPage("Success"); // Перенаправление! Реализация PRG
}
```

Upload (пересылка файлов с клиента на сервер)

- Форма должна иметь `enctype="multipart/form-data"`.
- Файлы пересылаются в теле запроса в формате `multipart/form-data`.
- Пересылаются файлы так называемыми чанками(это такой некоторый кусок файла, который пересылается потоком байт, вместе с текстовыми полями)
- На сервере файл принимается как `IFormFile`.

Пример реализации:

Разметка (`Upload.cshtml`):

```
@page
@model UploadModel

<form method="post" enctype="multipart/form-data">
```

```
<label>Выберите аватарку:</label>
<input type="file" name="Avatar" />
<button type="submit">Загрузить</button>
</form>
```

Бэкенд (Upload.cshtml.cs):

```
using Microsoft.AspNetCore.Http;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.RazorPages;
using System.IO;
using System.Threading.Tasks;

public class UploadModel : PageModel
{
    // Имя свойства должно строго совпадать с name="Avatar" в HTML
    [BindProperty]
    public IFormFile Avatar { get; set; }

    public async Task<IActionResult> OnPostAsync()
    {
        if (Avatar == null || Avatar.Length == 0)
        {
            ModelState.AddModelError("", "Файл не выбран или пуст");
            return Page();
        }

        // 1. Определяем путь, куда сохранить файл (например, в папку
        wwwroot/uploads)
        var folderPath = Path.Combine(Directory.GetCurrentDirectory(),
            "wwwroot", "uploads");

        // Гарантируем, что папка существует
        if (!Directory.Exists(folderPath))
            Directory.CreateDirectory(folderPath);

        // Получаем безопасное имя файла (преподаватели ценят очистку
        оригинального имени)
        var fileName = Path.GetFileName(Avatar.FileName);
        var filePath = Path.Combine(folderPath, fileName);

        // 2. Открываем поток и асинхронно копируем туда данные файла
        using (var stream = new FileStream(filePath, FileMode.Create))
        {
            await Avatar.CopyToAsync(stream);
        }

        return RedirectToPage("Success"); // Снова паттерн PRG
    }
}
```

```
}  
}
```

Вопрос 24. Razor Page: tag helpers, частичные представления (partial)

Tag helper - специальный инструмент(специальный класс), который выглядит как HTML-тег, но на самом деле тегом не является. В cshtml «настоящих» тегов нет - это тег-хелперы, замаскированные под теги HTML.

Понять, что это тег-хелпер, а не обычный тег, можно по особым атрибутам (например `asp-for`, `asp-append-version="true"`). Таким же образом это понимает и Razor Engine, перехватывая такой тег и обрабатывая на стороне сервера.

Важно понимать, что такие тэги не попадают в таком же виде клиенту, т.е. в браузерной разметке не будет никакого `"asp-for"` и тд.

```
@* Было в Razor: *@  
<a asp-page="/Catalog/Details" asp-route-id="5">Отредактировать</a>
```

```
@* Стало в браузере клиента: *@  
<a href="/Catalog/Details?id=5">Отредактировать</a>
```

```
<a asp-page="/Index">Главная</a>  

```

Частичное представление (partial) - отдельное cshtml-представление, которое можно вставить в другое представление. Нужно, чтобы сократить разметку и переиспользовать её; можно передать ему модель.

Внутри такого представления нет директивы `@page`, а так же не может обрабатывать HTTP-запросы напрямую.

Данный подход реализует подход DRY(don't repeat yourself). Т.е. один раз вынес и используешь потом много раз.

По умолчанию частичное представление наследует (`ViewData`) и модель родительской страницы. Но правильный подход — передавать туда конкретный объект, с которым partial должен работать.

Пример использования:

У нас есть модель карточки товара на родителе и цикл `foreach` :

```
@page  
@model CatalogModel
```

```
<div class="product-grid">
  @foreach (var item in Model.Products)
  {
    // Вызываем частичное представление для каждого товара в списке
    <partial name="_ProductCard" model="item" />
  }
</div>
```

Внутри файла `Shared/_ProductCard.cshtml` :

```
@model Project.Models.Product
@* Указываем, какой тип данных мы ожидаем на входе *@

<div class="card">
  
  <h3>@Model.Name</h3>
  <p>Цена: @Model.Price руб.</p>
  <button>Купить</button>
</div>
```

Инструмент	Для чего глобально нужен?	Применяется к чему?	Пример
Layout	Глобальный внешний «каркас» (шапка, подвал). Страница вставляется <i>внутри</i> него.	Ко всей странице целиком	<code>_Layout.cshtml</code>
Partial View	Повторно используемый блок разметки, который вставляется <i>внутри</i> страницы.	К куску разметки (виджет, карточка)	<code>_ProductCard.cshtml</code>
Tag Helper	Логика на сервере для модификации стандартного поведения HTML-тегов.	К конкретному HTML-тегу	<code><a asp-page="..."></code>

Вопрос 25. MVC: определение, назначение, схема приложения, структура проекта

MVC (Model-View-Controller) - архитектурный паттерн, включающий три компоненты: модель (данные), представление (отображение модели), контроллер (обработка запросов, координация модели и представления). Плюс маршрутизатор - выбор контроллера и действия.

Главный принцип MVC — **Separation of Concerns (Разделение обязанностей)**. Каждый компонент занимается *строго своим делом*:

- **Model (Модель):** Содержит бизнес-логику, данные и правила их обработки. Модель «ничего не знает» про HTML, веб-запросы или CSS. Она просто умеет работать с данными (например, считывать из базы данных, валидировать поля, делать расчеты).
- **View (Представление):** Отвечает исключительно за визуальное отображение данных (UI). В ASP.NET Core это файлы `.cshtml` (синтаксис Razor). Представление получает готовые данные от контроллера и превращает их в HTML. Оно не должно делать никаких запросов к базе данных или выполнять сложную логику.
- **Controller (Контроллер):** Логический центр и «дирижер» приложения. Он принимает HTTP-запрос от пользователя, решает, какую логику из *Модели* нужно вызвать, забирает результат и передает его в нужное *Представление*.

Назначение

Упорядочить работу программистов. Главная задача - чтобы модель и контроллер были как можно более независимы: изменение в одном не влияет на другое. Это делает удобной разработку, а так же улучшает тестируемость кода(в целом цели те же, что и в MVVM)

Схема приложения

Запрос → middleware → маршрутизатор выбирает контроллер и действие → на каждый запрос создаётся контроллер → контроллер вызывает View (Razor Engine генерирует ответ) → response клиенту.

- **HTTP Request:** Клиент отправляет запрос на сервер (например, `GET /Catalog/Details/5`).
- **Middleware (Конвейер):** Запрос проходит через цепочку промежуточного ПО (логирование, авторизация, обработка ошибок).
- **Routing (Маршрутизатор):** Система маршрутизации анализирует URL и сопоставляет его с нужным контроллером и действием (Action). В нашем случае: контроллер `CatalogController`, метод `Details`, параметр `id = 5`.
- **Создание контроллера:** На *каждый* входящий HTTP-запрос движок ASP.NET Core создает **новый экземпляр** класса контроллера и внедряет в него зависимости через DI-контейнер. После отправки ответа этот экземпляр уничтожается.
- **Работа Action:** Метод контроллера обращается к слою Модели (например, запрашивает продукт с ID 5 из базы данных через Entity Framework).
- **Вызов View:** Контроллер передает полученный объект продукта в метод `View(product)`. Razor Engine берет шаблон, склеивает его с данными модели и компилирует в чистый HTML.
- **HTTP Response:** Готовый HTML-код отправляется обратно пользователю.

Структура проекта

- **Controllers** - контроллеры (классы с именем из двух частей: имя + суффикс Controller).
- **Models** - модели данных, контекст БД.
- **Views** - представления (.cshtml).
- **Program.cs** - настройка, таблица маршрутизации.

Файл Program.cs

В современных версиях .NET (начиная с .NET 6+) это единая точка входа, где настраивается всё MVC-приложение. Для работы MVC тут обязательно должны быть две вещи:

1. Подключение сервисов MVC в DI-контейнер:

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllersWithViews(); // Регистрирует контроллеры и движок Razor
```

2. Настройка таблицы маршрутизации (Routing): Указывает дефолтное правило, по которому строки из URL мапятся на контроллеры.

```
var app = builder.Build(); app.MapControllerRoute( name: "default", pattern:
"{controller=Home}/{action=Index}/{id?}" ); // Шаблон: ИмяКонтроллера/
ИмяЭкшена/НеобязательныйId app.Run();
```

Вопрос 26. MVC: маршрутизация, таблица маршрутизации, шаблоны маршрутов

Маршрутизация (Routing) — это процесс сопоставления входящего HTTP-запроса с определенным действием (Action) контроллера.

Таблица маршрутизации - набор шаблонов маршрутов; таблиц может быть несколько. По ним маршрутизатор выбирает контроллер и действие. Маршруты проверяются сверху вниз.

Когда приходит запрос, маршрутизатор берет URL и начинает сравнивать его с шаблонами в таблице **строго сверху вниз**, от первого правила к последнему.

- Как только находится **первое совпадение**, поиск прекращается. Маршрутизатор вырезает из URL нужные параметры и передает запрос контроллеру.
- Если запрос подошел под первое правило, до второго он даже не дойдет. Поэтому **более специфичные (узкие) маршруты всегда должны регистрироваться выше, чем общие (дефолтные)**.
- Если совпадений не найдено вообще, приложение возвращает ошибку 404 Not Found .

Шаблон маршрута

Обычно рассматривается 2-х или 3-х компонентный URI вида

`{controller}/{action}/{id?}` . Если ничего не указано - по умолчанию Home / Index.

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Action (действие) - публичный метод контроллера, доступный извне и вызываемый по URI.

Маршрутизацию можно задавать с констрейнтами (ограничениями), как и в Minimal API.

Чтобы маршрутизатор не передавал в метод заведомо некорректные данные (например, строковый текст вместо Id), в шаблонах можно задавать **констрейнты (ограничения)**. Они пишутся через двоеточие : после имени переменной.

Если URL не соответствует ограничению, маршрутизатор считает, что это правило не подходит, и идет искать совпадения дальше по таблице.

Популярные ограничения:

- **Типы данных:** `{id:int}` (только целое число), `{id:guid}` , `{isActive:bool}` .
- **Длина и диапазоны:** `{id:range(1, 100)}` (число от 1 до 100), `{username:minlength(3)}` (строка от 3 символов).
- **Регулярные выражения:** `{year:regex(@"\d{4}$")}` (строго 4 цифры года).

Пример с ограничением в таблице:

```
app.MapControllerRoute(
    name: "archive",
    pattern: "blog/{year:int}/{month:range(1,12)}",
    defaults: new { controller = "Blog", action = "Archive" }
);
```

Запрос `/blog/2026/5` — сработает. Запрос `/blog/hello/15` — проигнорируется (перейдет к следующему правилу).

Вопрос 27. MVC: контроллеры, жизненный цикл, фильтры, DI, обработка параметров, передача данных во View

Контроллер - класс, обрабатывающий запросы; имя состоит из двух частей - имя + суффикс Controller.

Жизненный цикл контроллера - контроллер создаётся на каждый запрос и умирает после каждого запроса.

Зачем так сделано? Это гарантирует **потокобезопасность (Thread Safety)**. Если бы контроллер был один (Singleton) на всё приложение, то при одновременном запросе от 100 пользователей их данные перепутались бы в локальных полях класса. А так у каждого запроса свой изолированный объект.

Цикл выглядит так:

1. Активатор контроллеров (`IControllerActivator`) создает экземпляр класса.
2. Через конструктор внедряются нужные сервисы (DI).
3. Отрабатывают фильтры и выполняется сам Action-метод.
4. Формируется ответ.
5. Контроллер уничтожается сборщиком мусора (`.NET Garbage Collector`). Если контроллер реализует интерфейс `IDisposable` , автоматически вызывается метод `Dispose()` для очистки неуправляемых ресурсов.

Фильтры контроллеров (Action-фильтры)

Фильтр - это класс, реализующий интерфейс. Некоторые фильтры работают и до, и после действия (action), некоторые только до, некоторые только после. В отличие от middleware (общий для всех точек), фильтр привязан к конкретному контроллеру/действию.

Middleware работают на уровне всего веб-сервера(читай "нашего приложения"), тогда как фильтры работают на уровне и в контексте MVC. Мидлвари не знают про MVC, контроллеры, экшены или параметры C#, они видят только сырой HTTP запрос, а фильтры могут видеть какой вызывается метод, какие в него переданы аргументы и в целом могут видеть метаданные C#, а так же напрямую подменить результат.

Типы фильтров (Конвейер фильтров):

Фильтры выполняются в строгом порядке, образуя внутренний конвейер:

1. **Authorization Filters** — Проверяют права пользователя (например, атрибут `[Authorize]`).
2. **Resource Filters** — Работают до привязки модели. Удобны для кэширования страниц.
3. **Action Filters** — Окружают непосредственно сам метод экшена. Метод `OnActionExecuting` вызывается до выполнения экшена (можно проверить валидность модели), а `OnActionExecuted` — после (можно модифицировать результат).
4. **Exception Filters** — Срабатывают, только если внутри экшена произошло необработанное исключение (ошибка).

5. **Result Filters** — Окружают генерацию результата (например, выполнение Razor View).

Применение DI в контроллерах (два способа)

- **Через конструктор** - сервис передаётся параметром конструктора (основной способ).

```
public class CatalogController : Controller
{
    private readonly IProductRepository _repository;

    // Сюда DI-контейнер сам передаст нужную реализацию
    public CatalogController(IProductRepository repository)
    {
        _repository = repository;
    }
}
```

- **Service Locator** - через `HttpContext.Services` можно вытащить любой сервис (Антипаттерн!!!).

```
public IActionResult Details(int id)
{
    var repo = HttpContext.RequestServices.GetService<IProductRepository>();
    var product = repo.GetById(id); return View(product);
}
```

От автора: Есть еще промежуточный легитимный способ — атрибут `[FromServices]` перед параметром конкретного метода, если сервис нужен только в одном экшне из десяти.

Обработка параметров

Параметры привязываются к параметрам action-метода (привязка модели по именам), как в Minimal API.

```
public IActionResult Edit(int id, string name) { ... }
```

Если в URL придет `?id=5&name=Tom`, переменные автоматически заполнятся. Движок также умеет собирать сложные объекты (классы): если передать класс `User`, он сопоставит свойства `User.Name`, `User.Age` с соответствующими полями формы.

Способы передачи данных во View

- **Модель** - return View(model) - передаём объект модели.
- **ViewData** - коллекция «ключ-значение» в рамках одного запроса (доступ через индексатор).
- **ViewBag** - та же коллекция, что ViewData, но доступ через свойства (динамически) (т.е. ViewBag являешь лишь динамической оберткой над ViewData и соответственно позволяет обращаться через точку без приведения типов, но при этом вычисляется только в рантайме(читай "во время выполнения").

Акцент преподавателя: ViewBag и ViewData - это одна и та же коллекция, просто разные способы доступа.

Вопрос 28. MVC: представление - жизненный цикл, DI, макетирование, @Razor-разметка

Представление (View) - cshtml-файл, который вызывается из контроллера и генерирует разметку ответа.

Жизненный цикл представления

При первом обращении cshtml компилируется, создаётся C#-объект, который генерирует разметку в body ответа. При последующих обращениях используется уже созданный объект (новой генерации нет). При перезагрузке сервиса цикл начинается заново. Будем считать, что компиляция происходит в runtime.

DI в представлениях

Зависимость внедряется директивой @inject прямо в cshtml.

Макетирование и Razor-разметка

Действуют те же директивы Razor (@model, @using, @inject, @{}, @if, @foreach) и то же макетирование (Layout, @RenderBody, @RenderSection, @section), что и в Razor Page. См. вопросы 20 и 21.

Вопрос 29. MVC: встроенные и пользовательские Html-helpers

Html-helper - метод (по сути статический помощник), который генерирует кусочек HTML-разметки; нужен, чтобы сократить разметку.

- **Встроенные Html-helpers** - уже готовые методы объекта Html: Html.ActionLink(...), Html.TextBoxFor(...), Html.BeginForm(...).
- **Пользовательские Html-helpers** - свои методы-расширения, которые мы создаём сами, чтобы переиспользовать разметку.

```
@Html.ActionLink("Главная", "Index", "Home")
@Html.TextBoxFor(m => m.Name)
```

Акцент преподавателя: Этот вопрос рассказан скороговоркой - преподаватель сильно придирается не будет. Достаточно сказать, что это такое и для чего применяется.

Вопрос 30. Web API: понятие REST, назначение, схема, реализации, особенности контроллеров API

Акцент преподавателя: Этот вопрос вынесен на следующий семестр («Web API не трогаем»). Ниже - краткая база на случай общего вопроса.

REST (Representational State Transfer) - архитектурный стиль взаимодействия компонентов распределённого приложения. Представление - это URI (идентификатор ресурса), а управление ресурсом - глаголами HTTP (GET, POST, PUT, DELETE). Автор - Roy Fielding (2000).

Web API - приложение (по сути MVC), которое возвращает JSON или просто статус, без View. Web API - способ реализации REST-архитектуры. Часто это веб-служба, которую вызывают другие приложения.

Соответствие методов (CRUD)

- **GET** - select (получить, ресурс часто коллекция).
- **POST** - insert.
- **PUT** - update.
- **DELETE** - delete.

Два типа ресурсов

- Коллекция: /api/users.
- Элемент коллекции: /api/users/288.

Особенности контроллеров API

- Нет View - возвращается JSON или статус.
- Помечаются атрибутом [ApiController], наследуются от ControllerBase.
- Маршрутизация атрибутами [Route], [HttpGet], [HttpPost] и т.д.
- Общий префикс ресурсов /api/...

Недостатки REST

- Нет общепризнанного стандарта RESTful. (Кто как хочет так и ...)
- Не все браузеры поддерживают полный словарь методов (PUT, DELETE) - часто только GET и POST. (Имеется в виду не обычные браузеры типа Chrome(и оные) или Firefox, а всякие WebView в приложениях, браузеры для людей с ограниченными возможностями и тд)

- Неоднозначность кодов состояний.(См "Статус код 418" в гугле)